Querying a Codebase with NVIDIA Auto Ontology

Will Cook

4 November 2026

A source-to-SQL adapter, demonstrated on Auto Ontology's own repository

Abstract

Can a database agent answer questions about software if the source is represented as tables? We built **code-ontology-bridge**, a Python extractor that turns source files, symbols, calls and existing docstrings into relational data and a semantic model for NVIDIA Auto Ontology. Applied to Auto Ontology's own repository, the bridge let its agent recover a caller chain that includes a function passed to `asyncio.to_thread`. A complete run of 22 author-written questions produced 21 tables matching the references; 20 also had substantive explanations. The run used DeepSeek v4.1 Flash, NVIDIA embeddings and reranking, explicit output requirements and a public-model compatibility patch. The contribution is a working adapter and an inspectable experiment on 473 files and 5,319 symbols. The questions informed development, so these results do not establish an accuracy advantage over schema-only querying or an agent reading source directly.

1. The question and the contribution

Suppose a developer changes `resolve_semantic_fks`, the function that resolves semantic foreign keys in Auto Ontology. Which callers reach it? Searching for its name finds an immediate caller. Reaching the scheduler also requires following a wrapper and recognizing a function passed as an argument. The question spans several files and two kinds of relationship: direct calls and callback references.

A source map can record those relationships once and make them available to SQL. NVIDIA Auto Ontology already has the other half of this workflow: an agent that retrieves semantic context, writes SQL and executes it against a connected database. This project connects the two. Instead of asking Auto Ontology about a sales database, we ask it about a database of its own implementation.

The work has three parts:

- **A source adapter.** A standalone extractor records Python source facts and exports DuckDB rows, model YAML and an evaluation dataset using Auto Ontology's existing contracts.
- **An exercised integration.** A compatibility patch configures public model clients and exposes an existing SQL routing option at the chat boundary. The retained run connects ingestion, model import, semantic compilation and question answering.
- **An inspectable evaluation.** All 22 requests, generated SQL statements, result tables, scoring rules and final explanations are retained, including the two answers that fail different checks.

The source-map approach comes from **CodeMap**, Will Cook's earlier source-navigation tool. This bridge adapts that approach into an installable package; it does not depend on CodeMap's private interface. NVIDIA supplies the database agent and semantic layer. The bridge supplies the source-derived data and its model.

2. How the source reaches the agent

Auto Ontology needs both a live database to query and semantic context that explains the database. The bridge exports them separately. Figure 1 follows the two paths to SQL execution.

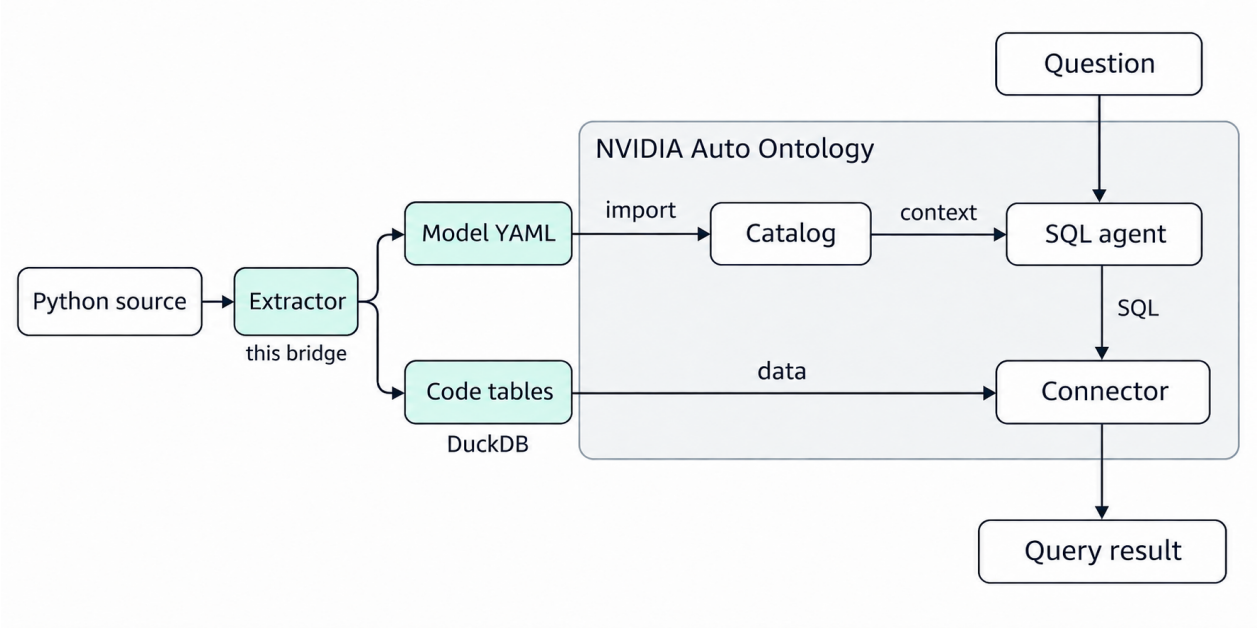


Figure 1: Two exports serve different purposes. DuckDB supplies source-code rows; model YAML supplies descriptions and relationships through import and compilation. The outlined components belong to Auto Ontology. The extractor is this bridge; the catalog is the semantic catalog and the connector is the existing DuckDB connector. This simplified query path omits ingestion services, model providers and answer narration.

The **DuckDB export** contains eight tables: files, docstring headings, symbols, class bases, calls, imports, test links and a source snapshot. Two views summarize incoming calls and public symbols without test associations. Auto Ontology reads these through its existing DuckDB connector.

The **model YAML** describes tables, terms, attributes and joins. It declares keys that the pinned DuckDB connector does not report. The live runner first ingests the database, maps the exported catalog identifiers to the server's identifiers, imports the model and compiles the semantic layer. Importing YAML alone does not create a queryable database.

A third export supplies DDL, CSV data and reference questions in NVIDIA's evaluation format. It provides a route toward a controlled comparison, but the full upstream benchmark harness was not run for this study.

What is a row, and what is a definition?

The extractor uses Python's AST and preserves source paths and line ranges. A function becomes a row in `code.symbols`. Its docstring becomes a value in `summary`. A call becomes a row in `code.calls`, with identifiers for its caller and, when resolved, its callee. Joining symbols to files lets a query recover locations or exclude test files.

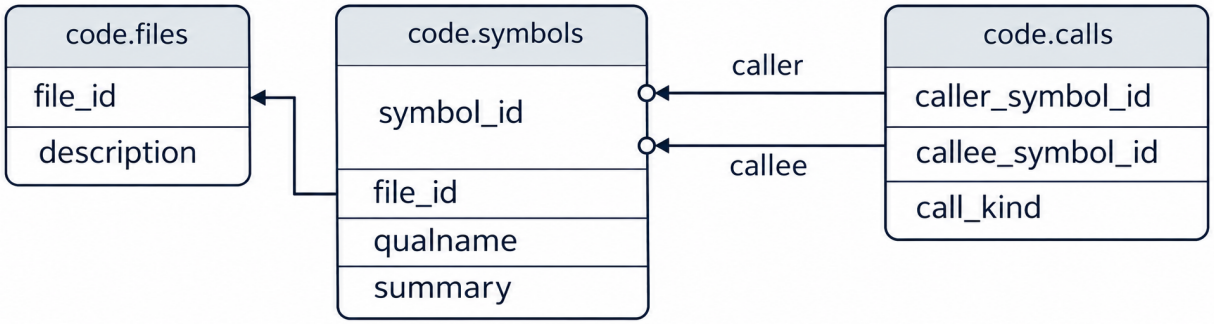


Figure 2: Selected columns show the joins used by a caller query. Each call references its caller and, when resolved, its callee; each symbol references its file. Descriptions of particular files and functions remain data values. The semantic model describes what the tables and columns mean.

This distinction keeps the model small: it defines a symbol as an entity type, rather than creating a separate semantic term for every function. Source descriptions are copied from existing docstrings. Missing descriptions stay empty; the extractor does not ask a model to invent them. The interface specification gives all columns, identity rules and resolution categories.

What changed upstream?

The data connector, model import and SQL pipeline already existed. A separate twelve-file compatibility patch configures public model clients, forwards the existing `calculation_only` option through the chat API and extends a placeholder validator. It adds no code-specific connector, SQL engine or query planner.

Fixed SQL routing matters here because a question such as "What does this module do?" can sound like a metadata question. In this dataset, the answer is a value in a table. The patch selects the row-querying path for every question in the run. Its documentation and validation bind the patch to 611 passing tests in the upstream query, chat and utility scopes. That is scoped validation, not the full upstream test suite or evidence of upstream acceptance.

3. Following the caller chain

Question 11 asks which resolved caller symbols reach `resolve_semantic_fks`, directly or indirectly, excluding tests and reporting the shortest depth. Auto Ontology generated recursive SQL over `code.calls` and returned four callers.

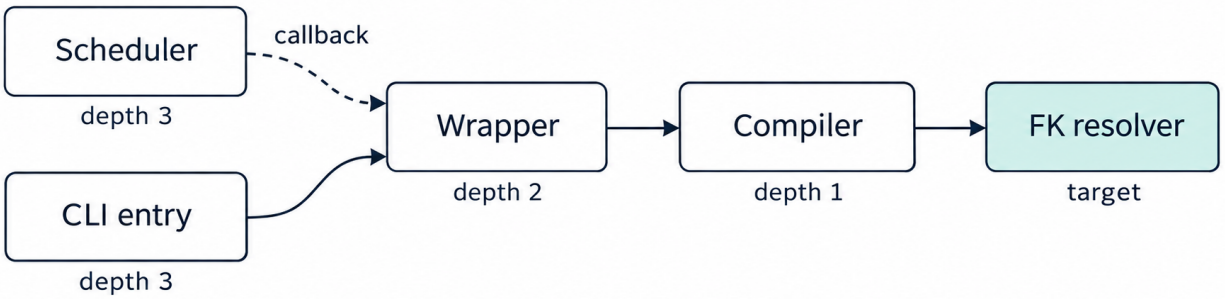


Figure 3: Four callers reach the foreign-key resolver through a wrapper and compiler. The table below identifies each source symbol. Arrows point from caller to callee, and SQL follows them in reverse. Solid edges are direct calls. The dashed edge records a callback passed to asyncio.to_thread, not proof that it ran.

Diagram label	Source symbol	Depth
Compiler	`_run_semantic_compilation`	1
Wrapper	`run_semantic_compilation`	2
Scheduler	`SemanticScheduler._run_once`	3
CLI entry	`main`	3

The target labeled “FK resolver” is `resolve_semantic_fks`; it is not a returned caller.

The scheduler passes `run_semantic_compilation` to `asyncio.to_thread`. The bridge records that argument as a callback reference, which connects the scheduler to the same chain as the ordinary caller `main`. The generated query follows reverse edges, avoids cycles and groups by full caller identity to select the minimum depth.

The saved response includes the request, intermediate events, final SQL and complete symbol identifiers. The request contains a hint about reverse traversal and callbacks; it is not an unaided discovery test. No saved SQL example was imported for this run. The answer establishes reachability within the extracted map. Dynamic calls can be missing, and a callback reference does not establish execution.

4. Evaluation: rows and explanations

All 22 questions were submitted once under protocol `explicit-output-v3`, with three concurrent requests and SQL routing enabled throughout. Auto Ontology’s bounded internal retries remained enabled. There were no selected replacement answers. The run began at 11:27:10 UTC and ended at 11:50:35 UTC on 4 November 2026.

The chat model was `deepseek/deepseek-v4.1-flash-1017` through OpenRouter. Embeddings used `nvidia/nemotron-3-embed-1b`; reranking used `nvidia/llama-nemotron-rerank-vl-1b-v2`. Eighteen questions contained authored hints. Every request specified required column names and types, with general guidance on distinct identities, caller kinds and numeric precision.

Reference SQL and answer values were used after the responses for scoring; they were not sent in the requests. Seven example SQL analyses remain in the distributed dataset but were excluded from the imported model.

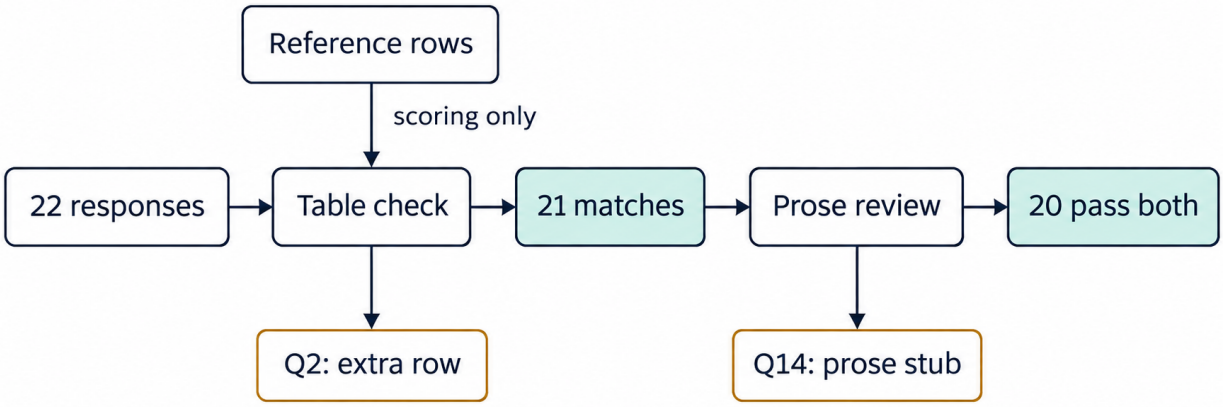


Figure 4: The two checks explain the reported counts: 21 tables match, and 20 of those answers also have substantive explanations. Reference rows enter scoring only. The branches identify the extra row in Q2 and the prose stub in Q14. Every answer was reviewed; this flow groups the outcomes of one development run.

The table scorer requires named, typed columns, full case-sensitive identities, correct row multiplicity and requested ordering. Integers match exactly; other numbers allow absolute transport error up to 1e-6. HTTP completion must succeed. An automatic prose check rejects empty responses, known placeholders and generic unrelated refusals. Review of every final query, table and explanation supplements that check.

Measure	Result
Completed requests	22 / 22
Tables matching all requested fields and rows	21 / 22
Matching tables with substantive explanations	20 / 22

Q2 returned an extra package. Asked what the semantic module does, the agent selected `module LIKE '%semantic%'`. That matched both `auto_ontology.semantic` and the server's semantic-compilation package. The latter had an empty description. The reference selects only the first package.

Q14 returned correct rows but unusable prose. Its seven resolution-statistics rows matched, but the explanation was just "thought". The automatic placeholder check missed it. Manual review therefore reduces the combined count from 21 to 20. Neither response has been edited or replaced.

Other successful questions found mutually calling packages, transitive import dependencies and a base class with ten non-test subclasses. Most explanations describe the query and its assumptions, leaving answer values in the accompanying table. "Substantive" does not certify a polished standalone narrative.

5. What the result establishes

The experiment establishes source-to-SQL querying through Auto Ontology, including a recursive caller query with a callback edge. It does not measure an accuracy improvement.

Earlier complete runs shaped the output contracts, identity guidance, caller wording and placeholder checks. Q11 was clarified to include all caller symbols; all 22 reference queries and answer rows stayed unchanged. The reported score uses only the final complete run. Earlier runs remain in Git history.

There is no independently authored held-out set, second repository, schema-only control or comparison with an agent navigating source directly. Matching rows on one database also does not prove query equivalence: an unnecessary filter can leave this snapshot's answer unchanged. Zhong, Yu and Klein (2020) evaluate SQL across test databases; this study's single-snapshot comparison has a narrower scope.

The map itself is incomplete. Of 31,178 direct call sites, 9,442 resolve to an internal callee and 11,794 are classified as dynamic attribute calls without one. Another 1,046 module-level calls are omitted from the call table. The small residual category named `unresolved` does not measure all missing edges. Callback records have lower confidence; test links reflect imports or matching filenames rather than measured coverage. Repeated qualified definitions, such as property getters and setters, receive distinct line-qualified identities, while ambiguous targets remain unresolved.

Queryable source representations are established techniques. CodeQL builds databases for source analysis, and CodexGraph lets language-model agents query code graph databases. The contribution here is the adapter to Auto Ontology's contracts and its demonstrated use on source-derived rows.

6. Reviewing and extending the integration

For an Auto Ontology maintainer, the work separates into three reviewable pieces:

Review question	Implementation or evidence
How do source facts enter the existing interfaces?	Extractor, exporters and interface specification.
What did the live service need?	Compatibility patch, with configuration, boundaries and scoped tests.
What did the agent actually produce?	Detailed results, linking every response, automatic score and manual review.

The next experiment should freeze independent questions, compare the same model with schema-only and semantic-layer context, and report which questions each condition fixes or breaks. A second repository would test transfer. Comparing against direct source navigation would test when building the map repays its cost. These comparisons remain unperformed.

Separate governance commands record certification history, annotate SQL and reconcile definitions. They do not enforce certified-only retrieval or prove which definitions the agent used. This evaluation makes no new claim about those commands.

The run used Auto Ontology `cdaf61e` and bridge `34285a2`. The manifest binds full revisions, model settings and frozen inputs. Follow the reproduction guide for extraction, compatibility tests or a new live run. Figure notes document the illustrations' sources and omissions.